Axioma

Concept and Architecture Rationale

What we used, where we used it, why we chose it, and what we are deliberately not using yet

Hackathon reference document

Current scope: Mode 1 for Python and Java DSA analysis | Future direction: Mode 2 repository review and verified repair

1. Executive summary

Axioma is an evidence-first code intelligence platform. It accepts a coding problem and a Python or Java solution, matches the problem to a compatible local catalog pack, executes the solution inside a restricted Docker worker, checks available examples and bounded generated cases, estimates complexity using visible signals, and returns a report that explains what happened and how strong the evidence is.

The project is intentionally in an early development stage. Mode 1 is working around deterministic rules, language-specific heuristics, measurements, and structured evidence. There is no trained ML model or LLM in the current workflow. The larger product direction is Mode 2: repository-level review, risk ranking, visible patch proposals, and regression verification under human control.

Core product promise: show what was checked, how it was checked, what the signals suggest, and what remains uncertain.

This document is designed to answer the question judges often ask: why did the team choose this concept or technology instead of another one?

2. Product problem and design response

Problem we faced	Design response	Why this response
Pass or fail is too shallow	Return individual evidence claims and limitations	A user can inspect coverage instead of trusting a single verdict.
Submitted code is untrusted	Execute outside the API process in a restricted worker	A crash, timeout, or exploit attempt should not take down the request process.
Catalog matching can be wrong	Reject ambiguous or low-confidence matches	A wrong match would invalidate all later test results.
Generated inputs may lack an oracle	Label provenance and distinguish robustness from correctness proof	An input without a trusted expected output cannot prove semantics.
Complexity signals are incomplete	Expose method, estimate, confidence, and disagreement	The tool can abstain or qualify rather than invent certainty.
Optimizations can change behavior	Validate candidates before measuring them	A faster candidate is useful only if it preserves the tested behavior.
Evaluation can leak	Freeze fresh reviewed cases before scoring	Tuning and final evaluation must remain separate.

Our reasoning is to narrow the first product boundary until it is testable, then expand only when new evidence supports the expansion.

3. Architecture and technology choices

The architecture is a modular monolith with a separate worker. This keeps local development understandable while preserving a clean boundary between request coordination and untrusted execution.

Layer	Chosen technology	Where it is used	Why this choice
Frontend	React 19 + TypeScript + Vite	Submission form, job stream, report UI	Typed components and fast iteration without introducing a heavier frontend framework.
API	Python + FastAPI + Pydantic	Validation, routes, job creation, report contracts	Python matches the analysis domain; FastAPI and Pydantic give explicit request and response contracts.
Persistence	SQLAlchemy + Alembic	Jobs, events, tests, reports, migrations	Keeps schema changes explicit and supports SQLite locally and PostgreSQL when needed.
Database	SQLite locally; PostgreSQL supported	Queue state and durable reports	SQLite reduces local setup; PostgreSQL provides a stronger shared development path.
Queue	Database-backed worker queue	Queued, running, recovery, cancellation	Avoids a second queue service before scale requires it while still supporting durable state and leases.
Live updates	Server-Sent Events	Browser receives job status changes	The server owns state and the browser mainly needs one-way updates with reconnect support.
Execution	Docker Linux containers	Python and Java sandbox	Separates code execution from the API process and enables resource and network restrictions.
Quality	Pytest, Ruff, Vitest, Oxlint	Services, evaluator, frontend	Covers behavior, linting, and the typed UI without adding unnecessary dependencies.

4. Why we are not training or calling a model in Mode 1

Current truth: Mode 1 does not use a trained ML model or an LLM. Its outputs come from catalog matching, deterministic input generation, language-specific static heuristics, bounded measurements, and rule-based optimization proposals.

Why not train a model yet?

- The project does not yet have a sufficiently large, reviewed, task-specific dataset for the questions we want to answer.
- A model would need a clear target such as complexity class, defect risk, or explanation quality. “Use AI” is not a task definition.
- The evaluation must prevent leakage by splitting by solution, repository, problem family, or time where appropriate.
- A baseline is required. If a simple rule or static tool performs as well, a model may not add enough value to justify its cost and opacity.
- Model versioning, calibration, failure analysis, privacy handling, and reproducibility would add responsibilities before the current evidence gate is complete.

Why use deterministic analysis first?

- It is inspectable: the report can name the detected loop, recursion, sort, collection operation, or recognized shape.
- It is reproducible: the same code, inputs, runtime, and method can produce the same class of evidence.
- It makes abstention practical: unsupported patterns can return unknown instead of a confident guess.
- It creates a baseline and evaluation harness for any future model to beat.

This is not an anti-ML position. It is a sequencing decision: first define the task and evidence, then add a model only where it produces measured value.

5. Future model plan: which model, for what, and why

If future evidence shows that a model would help, the team should choose the model by task, data, latency, privacy, and explainability requirements. The model should not replace executed evidence or silently change code.

Possible task	Candidate approach	Why this approach	Why not the alternative
Complexity classification	Start with rules and structured features; evaluate a small classifier only if coverage gaps remain	Structured features are easier to audit and compare with a baseline.	A general LLM may produce fluent but unsupported complexity explanations and can be hard to calibrate.
Defect-risk ranking in Mode 2	A supervised ranking model trained on reviewed repository changes	Ranking can prioritize human attention without claiming that the top item is definitely a defect.	A generative model alone does not provide a reliable ranking objective or evaluation denominator.
Patch explanation	Optional LLM or code model constrained to explain an existing diff	Explanation is a bounded assistance task that can cite the diff and checks.	Using a model to invent a patch before static and regression checks would increase risk.
Patch generation	Optional code model behind a visible diff and strict verification gates	The candidate remains a proposal and is judged by compilation and tests.	Autonomous modification or automatic commit would violate human-control and safety goals.
Natural-language report summary	Small optional language model with no authority over evidence fields	It can improve readability while preserving structured facts as the source of truth.	A model-generated score or confidence number could confuse interpretation with evidence.

Selection rule: use the simplest method that meets the measured task requirement. The structured report remains authoritative even when a model is added.

6. Execution, sandbox, and security concepts

The security design follows a boundary-first approach because submitted source, filenames, problem text, test output, and suggestions are untrusted inputs.

Concept	What we use	Why it matters	Current limitation
Process isolation	Run code in a worker container, not inside FastAPI	Keeps compilation and runtime failures away from the request process.	Containers share the host kernel.
Network isolation	Disable outbound network by default	Prevents submitted code from reaching internal services, metadata endpoints, or external systems.	Public deployment still needs defense-in-depth review.
Least privilege	Non-root process, read-only root, narrow mounts	Reduces what code can modify or access.	Misconfiguration can weaken the boundary.
Resource limits	Time, CPU, memory, process, output, temporary-disk, and wall-clock limits	Prevents infinite loops, forks, memory exhaustion, and output floods.	Limits need continuous testing across runtimes.
Cancellation and cleanup	Terminate process trees and clean temporary artifacts	Avoids orphaned work and leftover source or output.	Worker restart and cancellation remain important operational test cases.
Privacy	No external AI service in current Mode 1; local reports and browser-local history	Reduces source-code exposure during the early stage.	Authentication, hosted retention, and account deletion are not built yet.

The honest deployment statement is: suitable for local development and demonstration, not yet a public code execution service.

7. Testing, oracles, and evidence semantics

Axioma treats a test passing as evidence for that test, not as a universal proof. The report preserves the source and strength of each claim.

Evidence type	Example	What it supports	What it does not support
Executed	Imported sample or sandbox run	The submission produced the expected result for that configured case.	All possible inputs are correct.
Static	Loop depth, recursion, sort, binary-search signal	A visible code pattern suggests a possible complexity class.	The runtime cost of every helper or data structure.
Statistical	Timing, Python line or opcode events, mutation results	An observed trend or test-quality signal for the measured sample.	A mathematical asymptotic proof or a universal reliability guarantee.
Model	Future reviewed model output	Only the defined model task under its evaluation conditions.	Authority over executed evidence or a reason to hide uncertainty.
Unverified	Missing oracle, unsupported pattern, ambiguous match	A clear statement that the workflow could not support a conclusion.	A reason to invent a score.

Oracle logic: a generated input without an expected-output oracle can reveal crashes, timeouts, or disagreement, but it cannot independently prove semantic correctness. References and properties must also be treated as evidence with provenance, not unquestionable truth.

8. Complexity and benchmarking concepts

Complexity is handled as a layered estimate because static code signals and empirical measurements answer different questions.

Signal	Used for	Why chosen	Important limitation
Static AST or token analysis	Recognize loop nesting, recursion, sorting, collections, and selected algorithm shapes	Fast, inspectable, and reproducible.	Does not expand arbitrary helper costs or prove behavior for every input.
Python line events	Count traced source-line execution in selected function frames	Provides a repeatable work proxy for supported Python runs.	Not CPU instructions; tracing can be disabled and native work may be omitted.
Python opcode events	Count interpreter bytecode dispatches	More fine-grained than source lines for Python-level work.	Not total program work or hardware instructions.
Java JVM timing	Observe warmed reflective wall-time growth	Provides a runtime signal for Java at declared sizes.	Includes JVM and reflection overhead and depends on host conditions.
Space measurements	Estimate traced Python allocation or Java heap-pool increases	Adds a separate memory signal when positive observations exist.	Not process RSS; native or off-heap allocation can be missing.

Benchmarks are measurements, not proof of asymptotic complexity. The report must keep the selected sizes, input family, runtime, warm-up policy, repetitions, raw observations, and uncertainty visible.

9. Optimization logic

Optimization is intentionally narrow and rule-based in Mode 1. The product does not silently rewrite user code.

Step	Logic	Reason
Detect	Recognize a supported pattern such as a nested Two Sum search.	Avoid proposing changes for arbitrary code where the rule is not reliable.
Propose	Return a candidate source and explanation while keeping the original visible.	The user remains in control and can inspect the change.
Validate	Run the candidate against the same relevant samples and generated cases.	A performance idea is not useful if behavior changes.
Measure	Optionally compare observed signals at the same benchmark sizes.	A fair comparison needs the same input family and environment.
Report	Separate equivalence counts, measurements, and limitations.	No improvement claim should be inferred from a single noisy run.

If evidence is inconclusive, the correct output is “no measured improvement” or “not enough evidence,” not a fabricated percentage.

10. Mode 1 to Mode 2 direction

Stage	Scope	Why this order
Mode 1 current	Catalog-assisted Python and Java DSA analysis	Creates a controlled workflow for matching, sandboxing, testing, and evidence.
Reliability milestone	Fresh reviewed holdout, coverage analysis, failure categories, sandbox review	Builds the evidence needed before making broader accuracy or deployment claims.
Mode 2 future	Repository review, risk ranking, visible repair proposals, regression verification	Extends the same evidence discipline to real code changes without silent mutation.

Mode 2 workflow

- Ingest a repository, branch, commit, or pull request and identify supported build context.
- Run deterministic static checks, type or compile checks, and repository-aware rules with file and line provenance.
- Rank risks by evidence, reproducibility, affected path, and likely impact.
- Create a visible patch proposal or diff, never a hidden edit.
- Compile and run regression, generated, or property checks against the candidate.
- Report passed, failed, skipped, or inconclusive verification gates.
- Let the developer accept or reject the proposal.

Mode 2 is not an autonomous agent that silently changes repositories, and it is not a guarantee that every defect will be found. It is a controlled proposal-and-verification workflow.

11. What we will say about future ML or LLM use

The team can say: “Mode 1 does not use a trained ML model or an LLM. We intentionally built a deterministic baseline first so we can inspect evidence, measure failure modes, and define the tasks where a model would add real value.”

If a future model is introduced, the project should document:

- The exact task and output field the model owns.
- Data provenance, labeling process, and privacy constraints.
- A baseline without the model.
- A leakage-resistant split and frozen evaluation manifest.
- Coverage, conditional accuracy, calibration, abstention, and failure categories.
- Model version, prompt or feature version, runtime, and rollback behavior.
- What the model is not allowed to override, especially executed test evidence and security gates.

This keeps the product honest: future AI is a measured component inside an evidence system, not a replacement for evidence.

12. Decision summary for judges

Question	Answer
Why no model now?	No task-specific reviewed dataset, baseline, and fair evaluation are complete yet. Deterministic analysis is the correct first baseline.
Why Docker?	Submitted code is untrusted and must execute outside the API process with resource and network controls.
Why SQLite and PostgreSQL?	SQLite keeps local development simple; PostgreSQL supports a stronger shared deployment path.
Why a database queue?	It gives durable job state and recovery without adding a second infrastructure service too early.
Why SSE?	The browser needs one-way, reconnectable job updates and the server owns the state.
Why not claim proof?	Examples, generated tests, heuristics, and benchmarks each support limited conclusions.
Why Mode 2 later?	Repository review and repair need a stronger evaluation and security foundation than the early Mode 1 scope.

Recommended closing statement

Axioma is an early-stage but working foundation for evidence-based code intelligence. We start with a focused DSA workflow because it lets us validate the trust boundary and evidence model. We then move toward Mode 2 repository review and verified repair, but only with visible proposals, regression checks, human control, and measured limits. The project is ambitious in direction and conservative in claims.

Repository references used for this rationale: README.md, docs/api-contract.md, docs/threat-model-v1.md, docs/risk-register.md, docs/release-candidate-v1.md, docs/specifications/Axioma_PRD_v1.1.docx, docs/specifications/Axioma_TRD_v1.1.docx, and the current API, worker, frontend, and evaluator code.

13. Latest implementation updates

The current checkout now includes an optional model-assisted learning layer and stronger provenance for evidence. These features extend Mode 1 without replacing its deterministic core.

New capability	What it does	Important boundary
Optional Gemma 4 advisory	After a completed run, the user can ask Google's hosted Gemma 4 model for a short approach explanation and one learning suggestion.	Low-confidence model evidence only. It cannot execute code, alter test totals, alter complexity, or edit source.
Evidence review surface	The UI summarizes recorded checks, saved inputs, detected code signals, and documented claim limits.	It reports available evidence; it does not infer complete edge-case coverage.
Input previews and hashes	Test cases can show a bounded preview and SHA-256 hash of the saved input snapshot.	The preview is bounded; the hash identifies the full saved snapshot.
Benchmark replay	Completed jobs with matching persisted snapshots can rerun the exact benchmark inputs.	Inputs are replayed exactly; elapsed timings can differ across environments.
Problem-pack identity	Reports retain matched slug, title, and source identity.	Later review stays tied to the saved analysis rather than a newly edited problem.
Open-source scope	Application code is documented under MIT.	Local catalogs, databases, credentials, model weights, and generated reports remain excluded from Git.

14. Why the new model feature is optional

Gemma 4 is used as a teaching and explanation layer because language models can summarize a completed analysis in accessible language. It is not used as the authority for correctness or complexity because the project already has stronger structured evidence for those fields. The API reconstructs saved Axioma facts and labels the advisory separately, so a model claim cannot overwrite a test result.

The hosted path requires a server-side GEMINI_API_KEY and sends the saved problem, code, and bounded evidence summary to Google. Users should avoid sensitive code and review current provider terms. When the key is absent or the provider fails, the local deterministic workflow continues to work.

Repository references used for this rationale: README.md, docs/api-contract.md, docs/threat-model-v1.md, docs/risk-register.md, docs/release-candidate-v1.md, docs/gemma4-advisory.md, docs/specifications/Axioma_PRD_v1.1.docx, docs/specifications/Axioma_TRD_v1.1.docx, and the current API, worker, frontend, and evaluator code.